# LAPORAN

# IMPLEMENTASI STORED FUNCTION DAN DATABASE TRIGGER

*Disusun Untuk Memenuhi Tugas-5*

***Mata Kuliah : Prak. Rekayasa & Arsitektur Basis Data***

**Dosen Pengampu:**

**Cecep Kurnia Sastradipraja, S.Kom., M.Kom.**




**Disusun Oleh:**

Salsabila Oktapia 2507035

**Kelas B**

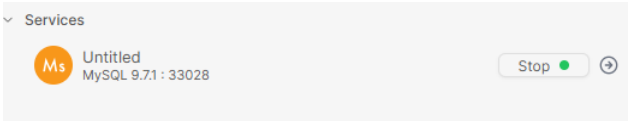
**JURUSAN ILMU KOMPUTER**

**PROGRAM STUDI SISTEM INFORMASI**

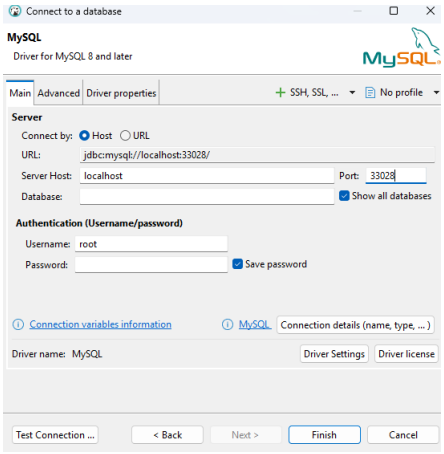
**INSTITUT TEKNOLOGI GARUT**

**2026**

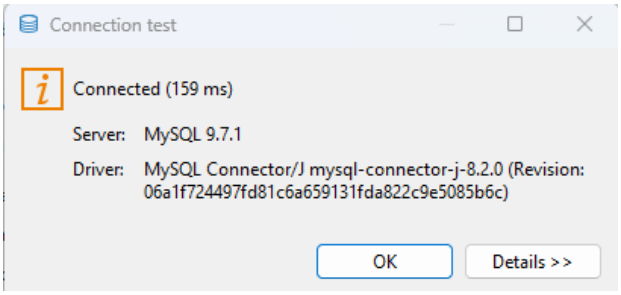
Mengaktifkan *service* MySQL 9.7.1 di DBngin pada port 33028 hingga berstatus *Running*.



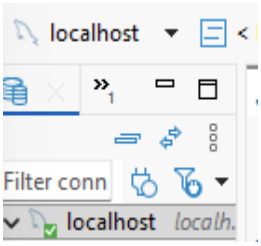

Membuat koneksi database MySQL baru di DBeaver dengan mengarahkan Server Host ke localhost dan Port 33028, lalu menguji koneksi (*Test Connection*) sampai muncul keterangan *Connected*.




Jika sudah terkoneksi klik OK,




Selanjutnya, sebelum membuat SQL nya kita harus klik kanan localhost, lalu pilih SQL editor dan pilih New SQL script.



1. **Tugas 1 : Function Perhitungan Pajak**

   Skrip SQL untuk membuat function hitung_pajak yang bertugas menghitung pajak sebesar 11% dari total transaksi.

```
DELIMITER //
CREATE FUNCTION hitung_pajak(
    p_total DECIMAL(14,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_total * 0.11;
END //
DELIMITER ;

SELECT 2000000 AS total, hitung_pajak(2000000) AS pajak; -- Hasil: 220000.00
SELECT 5000000 AS total, hitung_pajak(5000000) AS pajak; -- Hasil: 550000.00
```

   Hasil pengujian transaksi Rp2.000.000 dengan pajak 11% yang menghasilkan nilai Rp220.000.

```
SELECT 2000000 AS total, hitung_pajak(2000000) AS pajak; -- Hasil: 220000.00
```

| 123 total | 123 pajak |
|---|---|
| 2,000,000 | 220,000 |

   Hasil pengujian transaksi Rp5.000.000 dengan pajak 11% yang menghasilkan nilai Rp550.000.

```
SELECT 5000000 AS total, hitung_pajak(5000000) AS pajak; -- Hasil: 550000.00
```

| 123 total | 123 pajak |
|---|---|
| 5,000,000 | 550,000 |

2. **Tugas 2 : Function Total Akhir**

   Skrip SQL untuk membuat function hitung_total_akhir yang menghitung total belanjaan setelah dipotong diskon.

```
DELIMITER //
CREATE FUNCTION hitung_total_akhir(
    p_total DECIMAL(14,2),
    p_diskon_persen DECIMAL(5,2)
)
RETURNS DECIMAL(14,2)
DETERMINISTIC
NO SQL
BEGIN
    RETURN p_total * (1 - (p_diskon_persen / 100));
END //
DELIMITER ;
```

   Hasil pengujian total belanja Rp3.000.000 dengan diskon 15%, sehingga total bayar akhirnya menjadi Rp2.550.000.

```
-- Pengujian (Total Rp3.000.000, Diskon 15%)
SELECT hitung_total_akhir(3000000, 15) AS total_akhir; -- Hasil: 2550000.00
```

| 123 total_akhir |
|---|
| 2,550,000 |

3. **Tugas 3 : Trigger Pencatatan Perubahan Stok**

Menjalankan perintah USE untuk memilih database praktikum_function_trigger;.

```
USE praktikum_function_trigger;
```

| Name | Value |
|---|---|
| Updated Rows | 0 |
| Execute time | 0.003s |
| Start time | Thu Oct 29 06:47:10 WIB 2026 |
| Finish time | Thu Oct 29 06:47:10 WIB 2026 |
| Query | -- 3. Trigger Pencatatan Perubahan Stok |
| | USE praktikum_function_trigger |

Menghapus trigger lama bernama log_perubahan_stok_produk jika sebelumnya sudah pernah dibuat.

```
DROP TRIGGER IF EXISTS log_perubahan_stok_produk;
```

| Name | Value |
|---|---|
| Updated Rows | 0 |
| Execute time | 0.004s |
| Start time | Thu Oct 29 06:47:46 WIB 2026 |
| Finish time | Thu Oct 29 06:47:46 WIB 2026 |
| Query | DROP TRIGGER IF EXISTS log_perubahan_stok_produk |

Skrip SQL untuk membuat trigger baru yang otomatis mencatat riwayat perubahan stok barang ke dalam tabel log.

```
DELIMITER //
CREATE TRIGGER log_perubahan_stok_produk
AFTER UPDATE ON produk
FOR EACH ROW
BEGIN
    IF OLD.stok <> NEW.stok THEN
        INSERT INTO log_stok (
            id_produk,
            stok_awal,
            stok_akhir,
            keterangan
        )
        VALUES (
            NEW.id_produk,
            OLD.stok,
            NEW.stok,
            'Pembaruan stok manual'
        );
    END IF;
END //

DELIMITER ;
```

| Name | Value |
|---|---|
| Queries | 2 |
| Updated Rows | 0 |
| Execute time | 0.039s |
| Fetch time | 0s |
| Total time | 0.039s |
| Start time | 2026-10-29 06:48:12.157 |
| Finish time | 2026-10-29 06:48:12.216 |

Mereset stok produk Mouse kembali ke 25 unit dan menampilkan hasilnya di tabel produk.

```
-- 1. Reset stok Mouse (id_produk = 2) ke 25
UPDATE produk SET stok = 25 WHERE id_produk = 2;

select * from produk;
```

| | id_produk | nama_produk | harga | stok |
|---|---|---|---|---|
| 1 | 1 | Laptop | 7,500,000 | 10 |
| 2 | 2 | Mouse | 150,000 | 25 |
| 3 | 3 | Keyboard | 350,000 | 15 |
| 4 | 4 | Monitor | 1,800,000 | 8 |

Mengubah stok Mouse dari 25 menjadi 30 unit, lalu mengecek pembaruannya di tabel produk.

```
-- 2. Ubah stok Mouse dari 25 menjadi 30
UPDATE produk SET stok = 30 WHERE id_produk = 2;

select * from produk;
```

| | id_produk | nama_produk | harga | stok |
|---|---|---|---|---|
| 1 | 1 | Laptop | 7,500,000 | 10 |
| 2 | 2 | Mouse | 150,000 | 30 |
| 3 | 3 | Keyboard | 350,000 | 15 |
| 4 | 4 | Monitor | 1,800,000 | 8 |

Mengecek isi tabel log_stok untuk memastikan perubahan stok Mouse dari 25 ke 30 sudah tercatat secara otomatis.

```
-- 3. Cek catatan log
SELECT * FROM log_stok WHERE id_produk = 2;
```

| | id_log | id_produk | stok_awal | stok_akhir | keterangan | waktu_log |
|---|---|---|---|---|---|---|
| 1 | 1 | 2 | 25 | 23 | Pengurangan stok karena penjualan | 2026-10-29 06:41:32 |
| 2 | 2 | 2 | 23 | 25 | Pembaruan stok manual | 2026-10-29 06:50:45 |
| 3 | 3 | 2 | 25 | 30 | Pembaruan stok manual | 2026-10-29 06:56:51 |

## 4. Tugas 4 : Trigger Validasi Harga Produk

Menghapus trigger validasi_harga_produk jika sudah ada sebelumnya di database.

```
DROP TRIGGER IF EXISTS validasi_harga_produk;
```

| Name | Value |
|---|---|
| Updated Rows | 0 |
| Execute time | 0.01s |
| Start time | Thu Oct 29 07:00:21 WIB 2026 |
| Finish time | Thu Oct 29 07:00:21 WIB 2026 |
| Query | -- 4. Trigger Validasi Harga Produk |
| | DROP TRIGGER IF EXISTS validasi_harga_produk |

Skrip SQL untuk membuat trigger penolak otomatis jika ada orang yang mencoba menginput harga barang bernilai Rp0 atau kurang.

```
DELIMITER //
CREATE TRIGGER validasi_harga_produk
BEFORE UPDATE ON produk
FOR EACH ROW
BEGIN
    IF NEW.harga <= 0 THEN
        SIGNAL SQLSTATE '45000'
            SET MESSAGE_TEXT = 'Perubahan harga ditolak! Harga harus lebih besar dari 0.';
    END IF;
END //
DELIMITER ;
```

| Name | Value |
|---|---|
| Queries | 3 |
| Updated Rows | 0 |
| Execute time | 0.039s |
| Fetch time | 0s |
| Total time | 0.039s |
| Start time | 2026-10-29 07:01:19.162 |
| Finish time | 2026-10-29 07:01:19.207 |

Skenario A (Ubah harga Mouse jadi Rp200.000) berhasil dan diterima karena harganya lebih dari 0.

```
-- Skenario A: Mengubah harga Mouse menjadi Rp200.000 (Perubahan diterima)
UPDATE produk SET harga = 200000 WHERE id_produk = 2;
SELECT id_produk, nama_produk, harga FROM produk WHERE id_produk = 2;
```

| id_produk | nama_produk | harga |
|---|---|---|
| 2 | Mouse | 200,000 |

Skenario B (Ubah harga Mouse jadi Rp0) langsung ditolak oleh sistem dan memunculkan pesan error.

```
-- Skenario B: Mengubah harga Mouse menjadi Rp0 (Perubahan ditolak / Error)
UPDATE produk SET harga = 0 WHERE id_produk = 2;
```

```
SQL Error [1644] [45000]: Perubahan harga ditolak!
Harga harus lebih besar dari 0.
```

## 5. Tugas 5 : Analisis Integrasi Function dan Trigger

1. Apa perbedaan function hitung_total dengan trigger validasi_penjualan?
2. Apa dampaknya jika trigger pembaruan stok tidak dibuat?
3. Mengapa perubahan stok perlu dicatat dalam tabel log?
4. Bagaimana function dan trigger membantu menjaga integritas data?
5. Apa yang perlu diperhatikan ketika dua transaksi mencoba membeli produk dengan stok terbatas secara bersamaan?

**Jawaban :**

1. Function (hitung_total): seperti kalkulator. Tugasnya cuma menghitung total harga (harga × jumlah) lalu mengembalikan hasilnya kalau dipanggil.

   Trigger (validasi_penjualan): seperti satpam otomatis. Dia langsung jalan sendiri sebelum transaksi disimpan untuk mengecek apakah stok cukup, barangnya ada, dan harganya benar.
2. Stok barang di database tidak akan berkurang otomatis pas ada yang beli. Akibatnya, barang sudah laku tapi di sistem stoknya kelihatan masih utuh, sehingga pembeli lain bisa beli barang yang aslinya sudah habis.
3. Alasan Perubahan Stok Perlu Dicatat di Tabel Log Pencatatan di tabel log itu Sebagai catatan riwayat (bukti transaksi). Jika nanti ada barang hilang, rusak, atau salah hitung, kita bisa dengan mudah mengecek kapan dan kenapa stok tersebut berubah.
4. Function: Bikin hasil perhitungan (seperti harga total & diskon) selalu benar dan konsisten.

   Trigger: Mencegah masuknya data yang ngawur/salah secara otomatis (misalnya menolak jumlah beli $\le 0$ atau beli melebihi stok yang ada).
5. Penguncian Data (*Locking* / FOR UPDATE): Transaksi harus diproses antre satu per satu. Sistem harus mengunci stok untuk pembeli pertama sampai selesai, baru pembeli kedua bisa membaca sisa stoknya. Ini mencegah stok menjadi minus akibat rebutan.